

3-3) 탐색 알고리즘-1

- 탐색 알고리즘(Search Algorithm)

- 주어진 원소의 집합에서 특정 원소를 찾는 작업을 체계적으로 명시해놓은 것

- ① 순차 탐색 알고리즘:

원소 집합의 처음부터 하나씩 비교하며 탐색

- ② 이진 탐색 알고리즘:

원소 집합을 반으로 나누어 키(key)를 정하고
키와 특정 원소를 비교하여 특정 원소가 속하는
영역에 대해서만 탐색을 반복하는 방법

- 키를 정하는 규칙: $\left\lfloor \frac{i+n}{2} \right\rfloor$

(i : 시작 원소 인덱스, n : 마지막 원소 인덱스)

3-3) 탐색 알고리즘-2

- 다음에서 8의 인덱스를 찾는 적당한 탐색 방법은?

①	Index	0	1	2	3	4	5	6	7	8	9
	Value	1	9	3	7	3	10	6	8	2	4

②	Index	0	1	2	3	4	5	6	7	8	9
	Value	1	2	3	3	4	6	7	8	9	10

- ① 정렬되지 않았으므로 순차 탐색: 맨 앞부터 값을 비교하여 8번째(인덱스 7) 비교에서 탐색 완료.

- ② 정렬되었으므로 이진 탐색: $\left\lfloor \frac{i+n}{2} \right\rfloor = \left\lfloor \frac{0+9}{2} \right\rfloor = 4$,
인덱스 4의 4보다 크므로 $\left\lfloor \frac{5+9}{2} \right\rfloor = 7$,
인덱스 7의 8과 같음(2번째 비교에서 탐색 완료).

3-3) 탐색 알고리즘-3

▪ 순차 탐색 알고리즘

```
1 algorithm sSearch(int a[], int key, int n){  
2     int i;  
3     for i=0 to n-1  
4         if a[i]==key then return i;  
5     endif  
6     next i  
7     return -1;  
8 }
```

-시간복잡도: 평균 횟수 $= (1+2+\dots+n)/n = (n+1)/2$
 $f(n) = (n+1)/2 \leq (n+n)/2 = n \quad \therefore f(n) = O(n)$

3-3) 탐색 알고리즘-4

■ 이진 탐색 알고리즘

```
1  algorithm bSearch(int a[], int key, int n){
2      int left=0, right=n-1, mid;
3      while (left<=right)
4          mid=(left+right)/2;
5          if a[mid]<key then left=mid+1;
6          elseif a[mid]>key then right=mid-1;
7                      else return mid;
8      endif
9      endwhile
10     return -1;
11 }
```

3-3) 탐색 알고리즘-5

■ 이진 탐색 알고리즘

$$\text{- 시간복잡도: } f(n) = \begin{cases} c_1 & n = 1 \\ f\left(\frac{n}{2}\right) + c_2 & n > 1 \end{cases}$$

$$\begin{aligned} f(n) &= f(n/2) + c_2 = \{f(n/4) + c_2\} + c_2 \\ &= f(n/4) + 2c_2 = \{f(n/8) + c_2\} + 2c_2 \\ &= f(n/8) + 3c_2 = \dots = f(n/2^k) + kc_2 \\ &\leq f(1) + kc_2 \leq c_1 + c_2 \log n + c_2 \\ &\quad (\because 2^{k-1} \leq n < 2^k, k \leq \log n + 1 < k+1) \\ &\leq (c_1 + 2c_2) \log n = c \log n \\ \therefore f(n) &= O(\log n) \quad (\forall n \geq 2) \end{aligned}$$